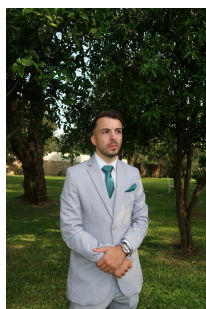


Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Programação Orientada aos Objetos

Ano Letivo de 2025/2026

Grupo 8 Domus Control - Home Assistant



Carlos Martins
A109507



Diogo Vieira
A109744



Tomás Machado
A104186

May 10, 2026

POO

Índice

1. Introdução	1
2. Arquitetura do Sistema	2
3. Estrutura Model-View-Controller	4
4. Descrição da Aplicação	6
5. Testes	13
5.1. Organização e Âmbito dos Testes	13
5.2. Testes de Stress e Robustez (SistemaStressTest.java)	13
5.3. Execução e Resultados	14
6. Conclusão	15
7. Referências Bibliográficas	16

1. Introdução

No âmbito da unidade curricular de **Programação Orientada aos Objetos** no ano letivo de 2025/2026, foi-nos proposto o desafio de desenvolver a aplicação **DomusControl**. Este sistema de gestão de dispositivos inteligentes é inspirado em soluções de automação doméstica como o **Home Assistant**.

O objetivo central deste projeto é a aplicação prática dos conceitos fundamentais do paradigma da Orientação aos Objetos — nomeadamente o encapsulamento, a abstração e a modularidade — através da criação de um ambiente de simulação capaz de gerir casas, divisões e diversos tipos de dispositivos conectados.

A aplicação **DomusControl** permite não só a gestão administrativa das habitações, como a criação de utilizadores, definição de divisões e associação de equipamentos, mas também a interação funcional com os mesmos. Para além do controlo individual, o sistema suporta mecanismos avançados de automação, tais como:

- **Automações:** Ativação de dispositivos com base em condições externas ou sensores.
- **Escalonamentos:** Regras de funcionamento baseadas em horários específicos.
- **Cenários:** Execução de instruções em série para múltiplos dispositivos em simultâneo (ex: “Sair de Casa” ou “Ver Cinema”).

Adicionalmente, o sistema integra funcionalidades de recolha de estatísticas de consumo e utilização, bem como a capacidade de salvaguarda e recuperação do estado da aplicação através de ficheiros binários. Este relatório descreve detalhadamente a arquitetura de classes adotada, as decisões de implementação e as funcionalidades desenvolvidas para cumprir os requisitos estabelecidos.

A camada de automação é composta pela classe abstrata `Acao`, pela interface `Condicao`, pelas classes `Automacao`, `Escalonamento` e `Cenario`. A `Automacao` associa uma condição a uma ação, sendo verificada periodicamente pelo sistema. O `Escalonamento` permite agendar ações com base em horários definidos, suportando execuções pontuais ou intervaladas. O `Cenario` agrupa uma lista de ações executadas em sequência, permitindo comportamentos compostos como “Sair de Casa” ou “Cinema”. As ações e condições são criadas através de métodos de fábrica estáticos que devolvem classes anónimas, evitando a proliferação de subclasses.

A classe `DomusControl` constitui o controlador central do sistema, mantendo todas as entidades centradas na lógica da aplicação, incluindo a verificação de permissões e a simulação da passagem de tempo através de um `LocalDateTime` interno.

Foi definido uma hierarquia de exceções de domínio com raiz em `DomusControlException`, com especializações para cada entidade não encontrada e `PermissaoNegadaException` para controlo de acesso.

Toda a hierarquia de classes implementa `Serializable`, permitindo guardar e restaurar o estado completo do sistema em ficheiro binário.

3. Estrutura Model-View-Controller

O sistema foi desenvolvido seguindo o padrão arquitetural MVC, que promove a separação de responsabilidades entre três camadas distintas: o Modelo (*Model*), a Vista (*View*) e o Controlador (*Controller*). Esta separação facilita a manutenção do código, permite testar cada camada de forma independente e torna o sistema mais extensível.

O *Model* é responsável por representar os dados e a lógica de negócio do sistema. É composto pelos pacotes `model` e `automacao`. O pacote `model` contém todas as entidades do domínio (*Dispositivo* e suas subclasses, *Divisao*, *Casa* e *Utilizador*), que encapsulam o estado do sistema e as regras associadas a cada entidade. O pacote `automacao` contém a lógica de automação (*Acao*, *Condicao*, *Automacao*, *Escalonamento* e *Cenario*), que é também parte do domínio do problema. O *Model* não tem qualquer dependência da *View*, sendo completamente independente da forma como os dados são apresentados ao utilizador.

A *View* é responsável por apresentar a informação ao utilizador e recolher as suas entradas. É composta pelo pacote `view`, com as seguintes classes:

- **ConsoleUI** responsável pela apresentação visual no terminal, com suporte a cores ANSI e adaptação dinâmica às dimensões da janela do terminal;
- **Menu** organiza a navegação entre os diferentes menus da aplicação, delegando ao `DomusControl` todas as operações sobre os dados;
- **InputValidator** centraliza e valida todas as entradas do utilizador, garantindo robustez face a entradas inválidas (por exemplo, texto quando se espera um número);
- **Main** ponto de entrada da aplicação, responsável por inicializar o controlador e arrancar a interface.

A *View* comunica exclusivamente com o `DomusControl`, nunca acedendo diretamente às entidades do *Model*.

O *Controller* é representado pela classe `DomusControl`, que funciona como intermediário entre a *View* e o *Model*. Esta classe centraliza toda a lógica de aplicação: criação e gestão de entidades, verificação de permissões de acesso, execução de automações e escalonamentos, simulação da passagem de tempo através de um `LocalDateTime` interno, e serialização/desserialização do estado completo do sistema para ficheiro binário. A *View* invoca os métodos do `DomusControl`, que por sua vez manipula os objetos do *Model* e devolve os resultados.

O fluxo típico de uma operação no sistema é o seguinte: o utilizador interage com a *View* (por exemplo, escolhe ligar uma lâmpada num menu); a *View* chama o método correspondente no `DomusControl`; o `DomusControl` localiza os objetos do *Model* relevantes, verifica permissões e executa a operação; o resultado é devolvido à *View*, que o apresenta ao utilizador.

Esta separação garante que uma eventual substituição da interface de consola por uma interface gráfica, por exemplo, não implicaria qualquer alteração nas camadas de domínio ou de controlo.

4. Descrição da Aplicação

Após inserir os seguintes comandos no terminal:

- make compile
- make run

O Utilizador terá o seguinte menu inicial:

```
diogo-vieira@Esquilo:~/uminho/ano2/Semestre2/P00/Projeto/2526-G008$ make compile
diogo-vieira@Esquilo:~/uminho/ano2/Semestre2/P00/Projeto/2526-G008$ make run
CONFIGURAÇÃO DE ARRANQUE:
1. Carregar estado anterior (Ficheiro Binário)
2. Iniciar novo Cenário de Teste (Fase 4)
Escolha:
```

Figura 2: Menu Inicial Utilizador

Ao iniciar a aplicação, o utilizador é confrontado com uma escolha de arranque: carregar o estado previamente gravado a partir de um ficheiro binário, ou iniciar com um cenário de teste gerado automaticamente. No encerramento, o estado completo do sistema é gravado no ficheiro estado_projeto.dat, garantindo a persistência de todos os dados entre sessões.

Após o arranque, o sistema apresenta um ecrã de login com a lista dos utilizadores registados. O utilizador selecciona o seu perfil pelo ID, não havendo sistema de passwords, o acesso é baseado em identidade. Ao introduzir 0, o programa encerra e grava o estado.

```

  LOGIN
  Bem-vindo ao Sistema DomusControl.
  Identifique-se para gerir a sua habitação inteligente.

  Utilizadores disponíveis:
  [1] Ana
  [2] Bruno
  [3] Carla

  Introduza o seu ID de utilizador
  (Ou digite 0 para encerrar o programa)

ID:
```

Figura 3: Menu de escolha do Utilizador

Após o login, o utilizador acede ao menu principal, que agrega todas as funcionalidades do sistema:

- **Gestão de casas** - acesso e administração das habitações
- **Criar Nova Casa** - registo de uma nova habitação no sistema
- **Automações** - visualização e controlo das regras automáticas
- **Estatística** - análise de consumo e utilização dos dispositivos
- **Escalonamentos** - programação de ações por horário
- **Cenários** - conjuntos de ações pré-definidas para situações do dia-a-dia
- **Mudar de Utilizador** - voltar ao ecrã de login
- **Sair e Gravar** - encerrar o programa com persistência de dados

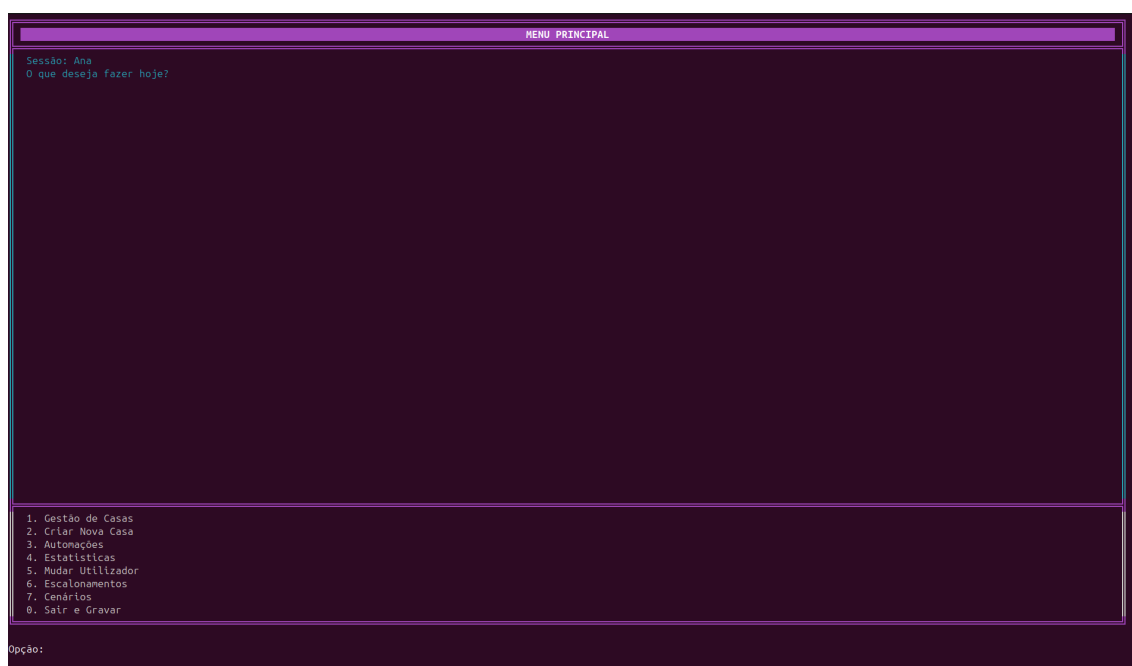


Figura 4: **Menu Principal**

Cada utilizador pode ter acesso a várias casas, com dois perfis distintos: **Administrador** ou **Utilizador**. O administrador tem permissões alargadas — criar e remover divisões, gerir utilizadores da casa, e eliminar a casa. O utilizador comum pode apenas consultar e interagir com os dispositivos.

Ao entrar numa casa, o sistema mostra o painel interno com a lista de divisões e o número de dispositivos em cada uma. É possível ver o estado global de todos os dispositivos da casa de uma só vez.

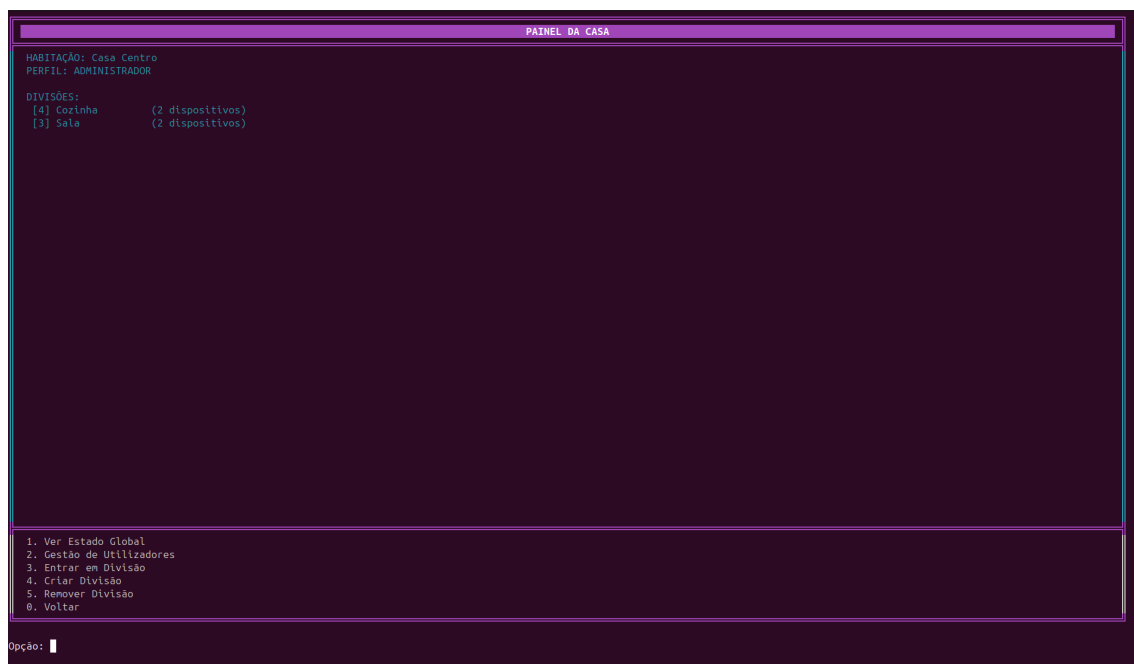


Figura 5: Menu Painel da Casa

Dentro de cada divisão, o utilizador pode visualizar todos os dispositivos instalados, com informação sobre o tipo, marca, modelo e estado atual. O sistema suporta sete tipos de dispositivos:

Dispositivo	Características específicas
Lâmpada	Cor da luz configurável
Tomada	Controlo de liga/desliga
Cortina	Nível de abertura (0–100%)
Coluna de Som	Volume configurável (0–100)
Portão de Garagem	Nível de abertura (0–100%)
Sensor de Água	Deteta presença de chuva
Sensor de Luz	Monitoriza nível de luminosidade

Todos os dispositivos registam o número de ativações e o tempo de uso acumulado, dados que alimentam as estatísticas do sistema.

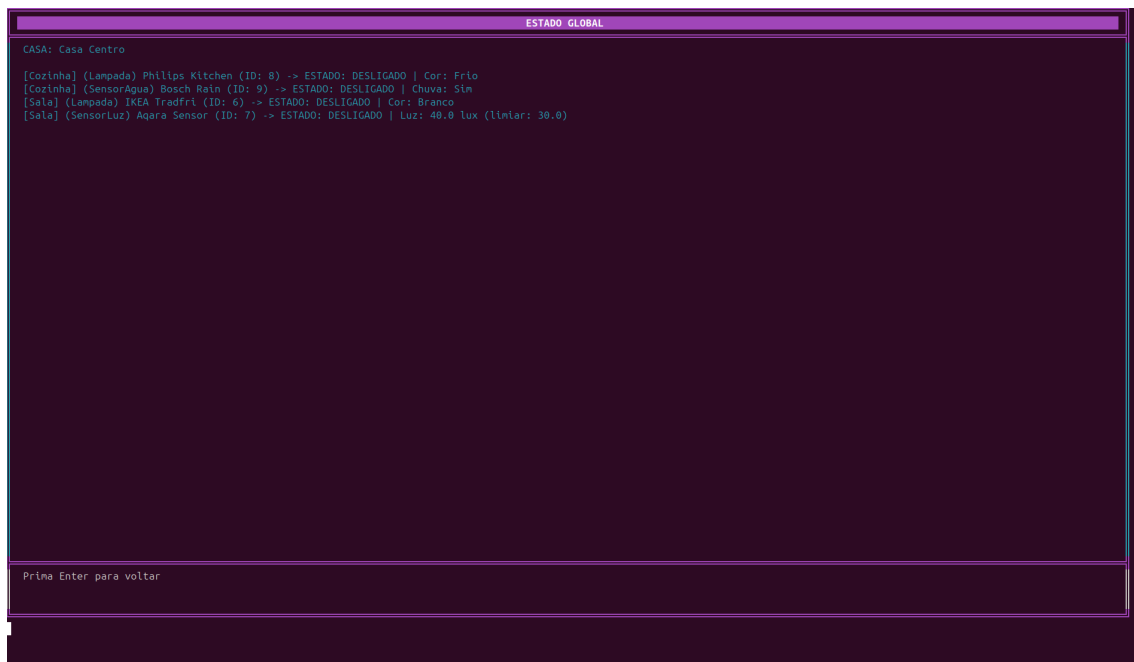


Figura 6: Estado Global Dispositivos

O administrador de uma casa pode, a qualquer momento, adicionar ou remover utilizadores, promovê-los a administradores ou retirar-lhes esse estatuto. O sistema impede a remoção do último administrador de uma casa, garantindo que esta nunca fica sem gestão.

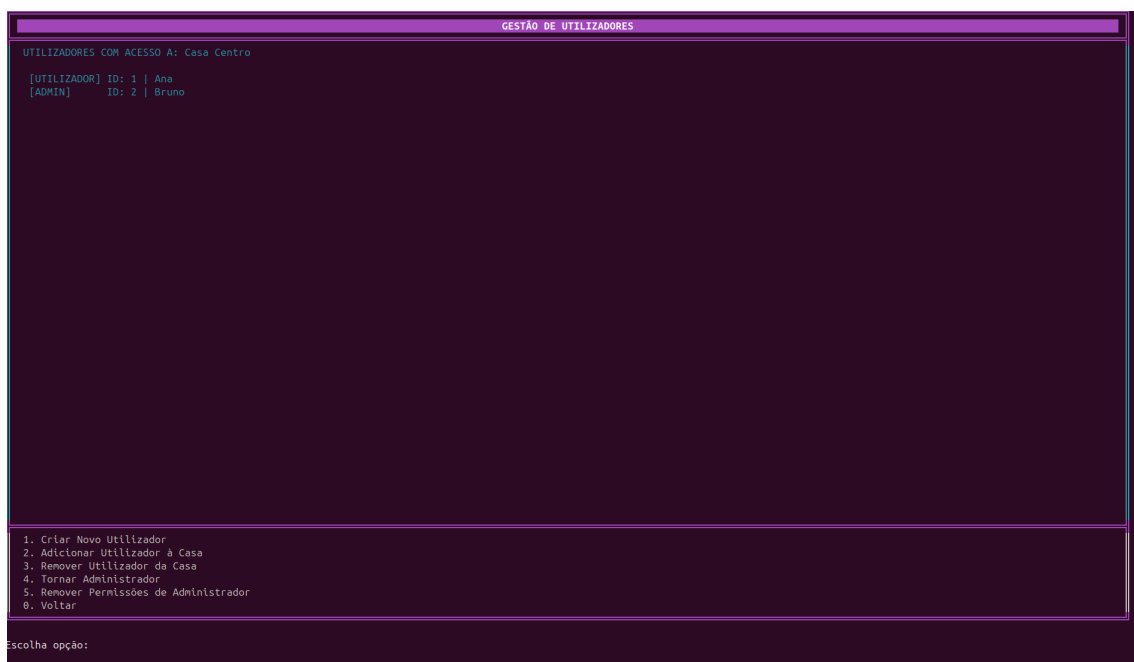


Figura 7: Gestão Utilizadores

O sistema inclui um motor de **automações** que reage automaticamente a eventos detetados pelos sensores. Ao criar uma casa, são automaticamente registadas quatro automações base:

- Fechar cortinas quando chove — ao detetar chuva (sensor de água), todas as cortinas da casa são fechadas

- Abrir cortinas quando a chuva para — ao detetar ausência de chuva, as cortinas são reabertas
- Modo Noite — ao detetar luminosidade baixa (sensor de luz), liga as lâmpadas
- Modo Dia — ao detetar luminosidade normal, desliga as lâmpadas

Cada automação pode ser ativada ou desativada individualmente. O menu de automações também permite simular manualmente a chuva ou a luminosidade baixa para testar o comportamento do sistema.

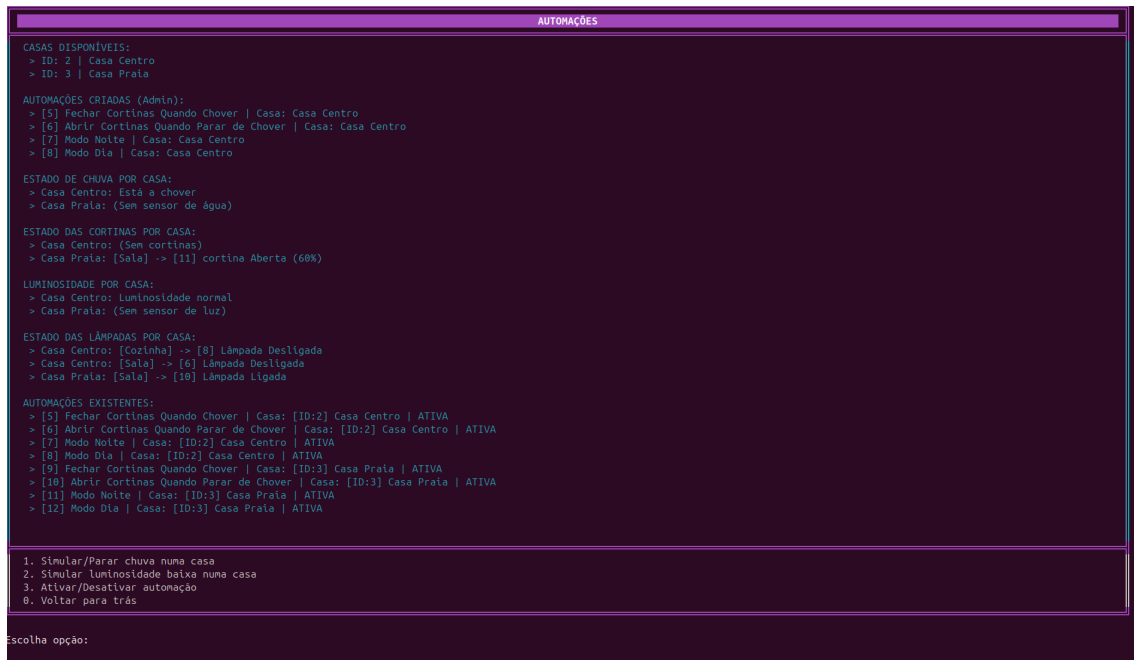


Figura 8: Menu Automações

Os **escalonamentos** permitem programar ações para ocorrerem em horários definidos, com suporte a hora de início e hora de fim. Cada escalonamento pode ser pontual (apenas ação de início) ou intervalado (ação de início + ação de fim), e o sistema garante que cada ação é executada no máximo uma vez por dia. O tempo é simulado internamente, podendo ser avançado manualmente.

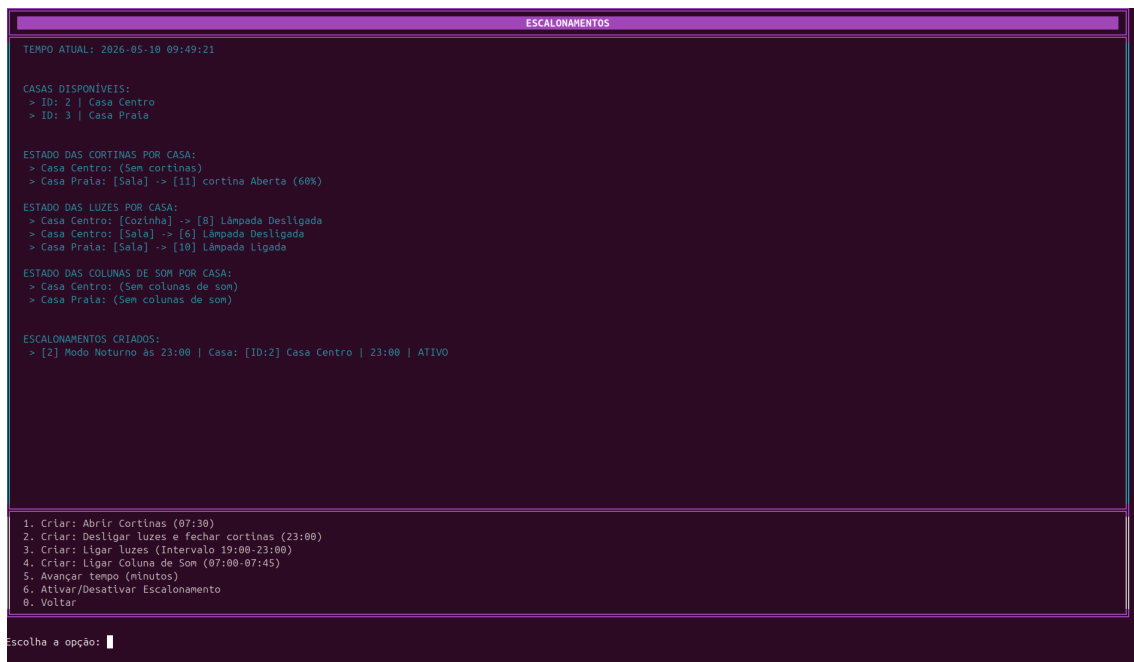


Figura 9: Menu Escalonamentos

Os **cenários** são conjuntos de ações predefinidas que podem ser aplicadas à casa com um único comando. O sistema disponibiliza cenários típicos como “Sair de Casa”, “Jantar com Amigos”, “Jantar Romântico”, “Cinema”, “Estudar”, “Deitar” e “Acordar”, cada um configurando o estado dos dispositivos de forma adequada ao contexto.

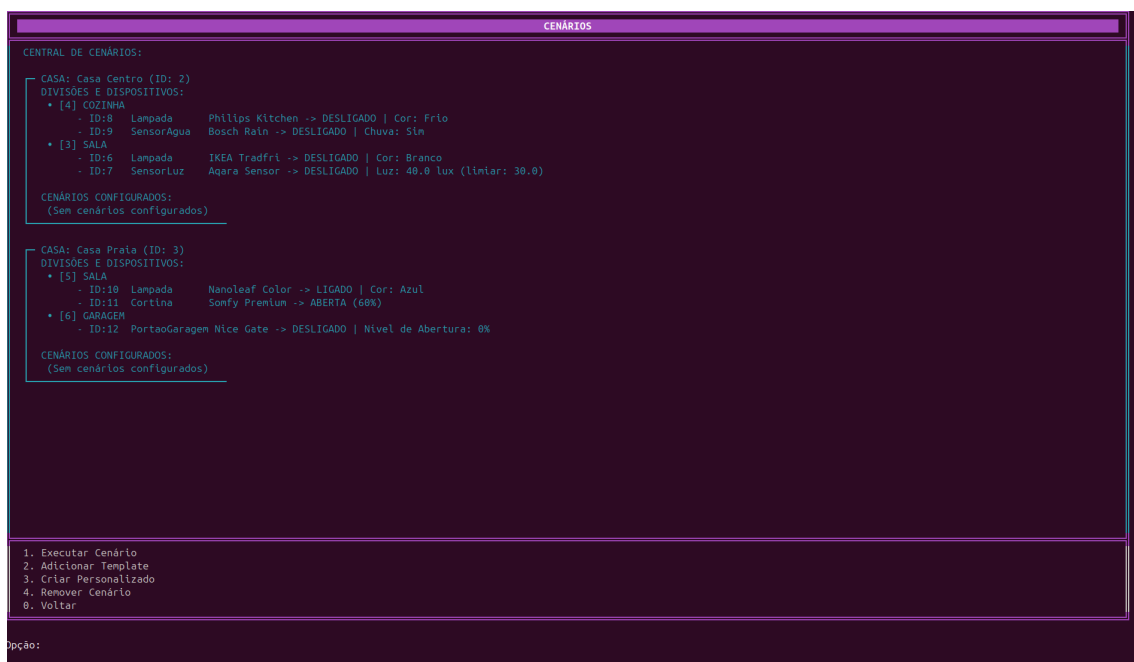


Figura 10: Menu Cenários

O módulo de estatísticas apresenta em tempo real:

- A casa que mais consome (soma do consumo dos dispositivos ligados)
- O top 3 de divisões com mais dispositivos instalados
- O top 3 de dispositivos por número de ativações
- O top 3 de dispositivos por tempo de uso acumulado

É também possível simular a passagem de 1 hora de tempo, incrementando o tempo de uso de todos os dispositivos ligados, ou consultar os dispositivos de uma casa específica.

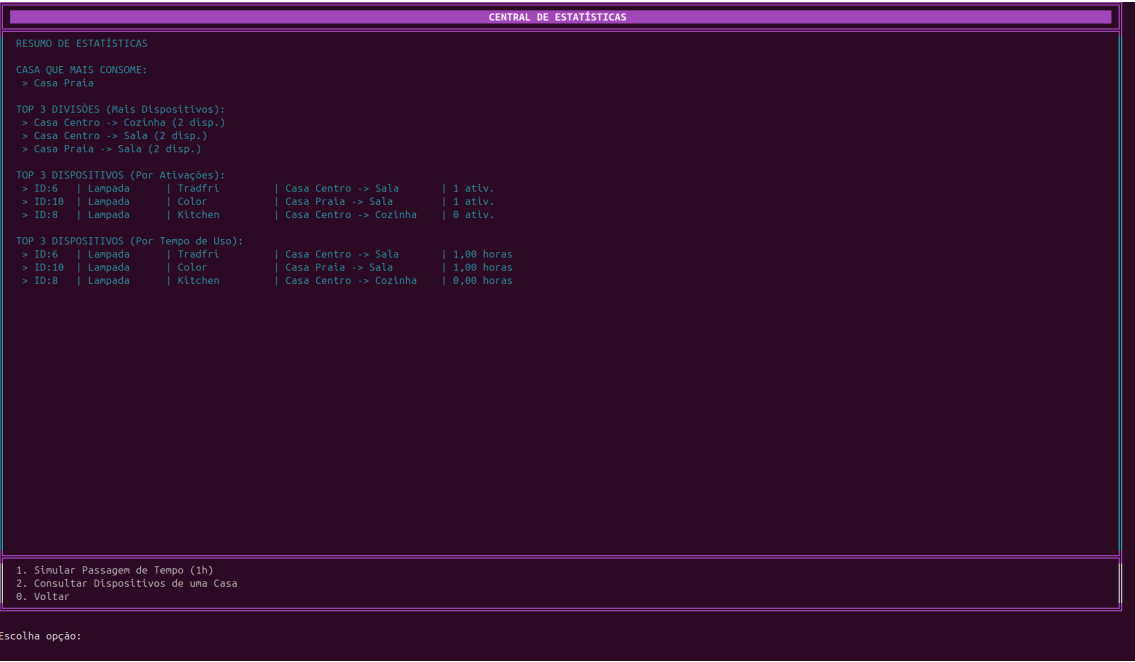


Figura 11: Menu Estatísticas

5. Testes

Para assegurar a fiabilidade, a integridade dos dados e o desempenho do sistema sob condições adversas, foi implementada uma suite de testes abrangente utilizando a framework **JUnit 5**. O projeto totaliza atualmente **153 testes unitários** distribuídos por 16 ficheiros.

5.1. Organização e Âmbito dos Testes

A suite de testes está organizada de forma modular para cobrir todas as camadas da aplicação:

- **Model (src/test/model)**: Validação das entidades base (Casa, Divisao, Utilizador) e da hierarquia de Dispositivo. Foca-se em garantir o funcionamento correto de getters/setters, métodos de comparação (equals/hashCode) e a integridade de clonagens profundas.
- **Automação (src/test/automacao)**: Com 72 testes, esta categoria valida o motor de regras, incluindo condições de disparo (como sensores de chuva e luz), execução de ações individuais, cenários pré-definidos e o agendamento temporal através de escalonamentos.
- **Controller (src/test/controller)**: Testes focados na fachada principal (DomusControl), assegurando que a lógica de negócio, a gestão de permissões e os fluxos complexos de criação de entidades funcionam conforme esperado.
- **View e Interface (src/test/view)**: Validação da interface de utilizador em console, testando a renderização visual de dashboards, constantes ANSI e a robustez do InputValidator no processamento de entradas do teclado.
- **Exceções (src/test/Exceptions)**: Garantia de que o sistema reage de forma segura a estados inválidos, lançando as exceções personalizadas adequadas (ex: CasaNaoEncontradaException) com as mensagens de erro previstas.

5.2. Testes de Stress e Robustez (SistemaStressTest.java)

Para elevar o nível de confiança na aplicação, foi desenvolvido um componente dedicado a testes de resiliência e fiabilidade:

- **Carga Extrema**: Simulação de **5000 utilizadores, 500 casas, 1000 divisões e 5000 dispositivos**. Este teste garante que operações como a passagem de tempo global funcionam sem degradação de desempenho ou erros de lógica.
- **Fuzzing e Entradas Anómalas**: Teste de resistência a inputs críticos, validando o tratamento de erros sem interrupção do serviço.
- **Stress de I/O e Memória**: Execução de ciclos intensivos de serialização (salvaguarda e carregamento do estado 1000 vezes consecutivas) e deteção de fugas de memória (memory leaks) através da criação e remoção massiva de objetos.

5.3. Execução e Resultados

A validação pode ser realizada de forma totalmente automatizada através do sistema de compilação do projeto:

```
1 make test
```

Sumário Final de Cobertura:

- **Total de ficheiros de teste:** 16
- **Total de métodos de teste:** 153
- **Estado Atual:** 153 testes bem-sucedidos / 0 falhas

```
Test run finished after 68017 ms
[      19 containers found      ]
[      0 containers skipped    ]
[      19 containers started   ]
[      0 containers aborted    ]
[      19 containers successful ]
[      0 containers failed     ]
[     153 tests found          ]
[      0 tests skipped         ]
[     153 tests started        ]
[      0 tests aborted         ]
[     153 tests successful     ]
[      0 tests failed          ]
```

Figura 12: Resultado da execução da suite de testes JUnit 5

6. Conclusão

O desenvolvimento do sistema DomusControl permitiu consolidar de forma prática os pilares fundamentais da Programação Orientada aos Objetos. Através da implementação de uma hierarquia de dispositivos baseada em classes abstratas e interfaces, foi possível explorar o polimorfismo e a herança, garantindo que o sistema fosse facilmente extensível para novos tipos de hardware sem comprometer a estrutura de dados existente.

A adoção do padrão arquitetural Model-View-Controller (MVC) revelou-se uma decisão crítica para a organização do projeto. Esta separação de responsabilidades não só facilitou a depuração durante o desenvolvimento, como garantiu que a lógica de negócio permanecesse independente da interface de utilizador. A implementação de mecanismos de persistência de dados via serialização assegurou que o sistema cumprisse os requisitos de continuidade, permitindo a salvaguarda e recuperação do estado completo da aplicação entre sessões.

O motor de automações, escalonamentos e cenários representou o maior desafio técnico do trabalho, exigindo uma gestão rigorosa do tempo simulado e da verificação de condições lógicas. A utilização de métodos de fábrica e classes anónimas para a definição de ações permitiu criar um sistema flexível que mimetiza com eficácia o comportamento de soluções reais de automação doméstica.

Em suma, o projeto atingiu os objetivos propostos na unidade curricular. O resultado final é uma aplicação robusta, modular e escalável, que demonstra como a abstração e o encapsulamento podem ser utilizados para modelar sistemas complexos de forma eficiente. Como trabalho futuro, o sistema encontra-se preparado para a integração de uma interface gráfica ou para a expansão da lógica de sensores para modelos de consumo energético mais granulares.

7. Referências Bibliográficas

- [1] E. Sciore, *Java Program Design: Principles, Polymorphism, and Patterns*. Apress, 2019.
- [2] JUnit Team, «JUnit 5 User Guide». [Em linha]. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>
- [3] Oracle Corporation, «Java Platform, Standard Edition 8 API Specification». [Em linha]. Disponível em: <https://docs.oracle.com/javase/8/docs/api/>
- [4] A. N. Ribeiro, «Slides da Unidade Curricular de Programação Orientada aos Objetos».
- [5] K. Nalawade, «How to Write Unit Tests in Java». [Em linha]. Disponível em: <https://www.freecodecamp.org/news/java-unit-testing/>
- [6] Oracle Corporation, «How to Write Doc Comments for the Javadoc Tool». [Em linha]. Disponível em: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>